МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе №3

Специальность АС-67/24

Выполнил:
Тюрина Т. А.
Студент 3 курса
Группы АС-67
Проверил:
Крощенко А. А.
доц. ИИТ,

Брест 2026

Цель работы: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Ход работы

Вариант 9

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 9

- `Glass Identification`
- Классифицировать тип стекла на основе его химического состава
- **Задания:**

1. Загрузите данные и стандартизируйте их

Код:

```
import pandas as pd

df = pd.read_csv('/content/glass.csv')

display(df.head())
print("Размер датасета:", df.shape)
print("Столбцы:", df.columns.tolist())
```

Результат:

	RI	Na	Mg	Al	Si	K	Ca	Ba	Fe	Type
0	1.52101	13.64	4.49	1.10	71.78	0.06	8.75	0.0	0.0	1
1	1.51761	13.89	3.60	1.36	72.73	0.48	7.83	0.0	0.0	1
2	1.51618	13.53	3.55	1.54	72.99	0.39	7.78	0.0	0.0	1
3	1.51766	13.21	3.69	1.29	72.61	0.57	8.22	0.0	0.0	1
4	1.51742	13.27	3.62	1.24	73.08	0.55	8.07	0.0	0.0	1

```
Размер датасета: (214, 10)
Столбцы: ['RI', 'Na', 'Mg', 'Al', 'Si', 'K', 'Ca', 'Ba', 'Fe', 'Type']
```

2. Разделите выборку

Код:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Признаки и целевая переменная
X = df.drop(columns=['Type'])
y = df['Type']

# Разделение: 75% — обучение, 25% — тестирование
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# Стандартизация признаков
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Обучающая выборка:", X_train.shape)
print("Тестовая выборка:", X_test.shape)
print("Количество классов:", y.nunique())
```

Результат:

```
Обучающая выборка: (160, 9)
Тестовая выборка: (54, 9)
Количество классов: 6
```

3. Обучите модели k-NN, Decision Tree и SVM

Код:

```
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

models = {
    "k-NN": make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=5)
    ),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": make_pipeline(
        StandardScaler(),
        SVC()
    )
}
```

```
for name, model in models.items():
    model.fit(X_train, y_train)
    print(f"{name}: обучение завершено")
```

Результат:

```
k-NN: обучение завершено
Decision Tree: обучение завершено
SVM: обучение завершено
```

4. Сравните производительность моделей с помощью `classification_report` (sklearn.metrics)

Код:

```
from sklearn.metrics import classification_report,
accuracy_score

for name, model in models.items():
    y_pred = model.predict(X_test)

    print(f"\n{name}")
    print(classification_report(y_test, y_pred,
zero_division=0))
    print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
```

Результат:

```
k-NN
              precision    recall  f1-score   support

           1       0.75      0.83      0.79        18
           2       0.71      0.79      0.75        19
           3       0.00      0.00      0.00         4
           5       0.50      0.67      0.57         3
           6       1.00      0.50      0.67         2
           7       0.88      0.88      0.88         8

    accuracy                           0.74        54
   macro avg       0.64      0.61      0.61        54
weighted avg       0.70      0.74      0.71        54

Accuracy: 0.741

Decision Tree
              precision    recall  f1-score   support

           1       0.80      0.89      0.84        18
           2       0.82      0.47      0.60        19
           3       0.33      0.50      0.40         4
           5       1.00      0.67      0.80         3
           6       0.17      0.50      0.25         2
           7       0.89      1.00      0.94         8

    accuracy                           0.70        54
   macro avg       0.67      0.67      0.64        54
weighted avg       0.77      0.70      0.71        54
```

```
Accuracy: 0.704

SVM
              precision    recall  f1-score   support

           1       0.67      0.78      0.72        18
           2       0.70      0.74      0.72        19
           3       0.00      0.00      0.00         4
           5       1.00      0.67      0.80         3
           6       0.50      0.50      0.50         2
           7       0.89      1.00      0.94         8

    accuracy                           0.72        54
   macro avg       0.63      0.61      0.61        54
weighted avg       0.67      0.72      0.69        54

Accuracy: 0.722
```

Код:

```python
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

for k in range(1, 16):
    model = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(model, X_train, y_train, cv=5)
    print(f"k = {k:2}: средняя точность = {scores.mean():.3f}")
```

Результат:

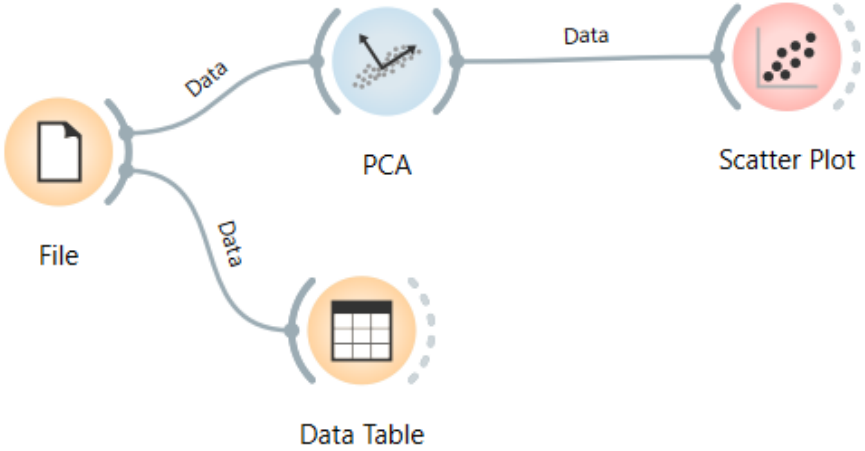
```
k =  1: средняя точность = 0.706
k =  2: средняя точность = 0.662
k =  3: средняя точность = 0.656
k =  4: средняя точность = 0.675
k =  5: средняя точность = 0.613
k =  6: средняя точность = 0.662
k =  7: средняя точность = 0.631
k =  8: средняя точность = 0.619
k =  9: средняя точность = 0.625
k = 10: средняя точность = 0.625
k = 11: средняя точность = 0.625
k = 12: средняя точность = 0.606
k = 13: средняя точность = 0.619
k = 14: средняя точность = 0.613
k = 15: средняя точность = 0.619
```

По результатам оценки на тестовой выборке модель k-NN показала точность 74,1%, SVM – 72,2%, а Decision Tree – 70,4%. Наибольшую точность продемонстрировала модель k-NN, а наименьшую – Decision Tree. С помощью classification_report были также сравнены показатели precision, recall и F1-score для классов.

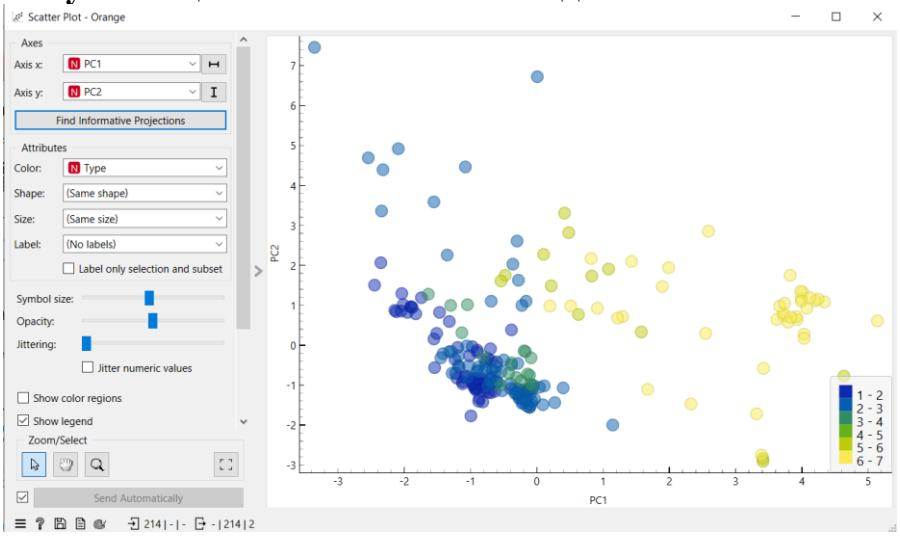
5. Укажите, какой класс модели определяют хуже всего, и предположите, почему

Хуже всего модели распознавали класс стекла 3. Модели k-NN и SVM не смогли правильно классифицировать ни один из четырёх объектов этого класса в тестовой выборке, показатель recall составил 0,00. У модели Decision Tree recall для класса 3 составил 0,50. Возможными причинами являются небольшое количество объектов этого класса в тестовой выборке и сходство химического состава стекла разных типов, из-за чего моделям сложно их различать.

Диаграмма, построенная в Orange



Визуализация типов стекла методом главных компонент (PCA)



Вывод: На практике сравнила работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научилась подбирать гиперпараметры моделей и оценивать их влияние на результат.